

Technical Security Analysis Portfolio

Reverse Engineering & Security Assessment of Obfuscated Local Data Storage in a Native C++ Mobile Game

Responsible Disclosure Notice

This document is prepared strictly for educational, security auditing, and defensive research purposes. The methodologies described herein demonstrate static analysis, pattern recognition, and architectural tracing used to identify weaknesses in client-side binary data storage and runtime data handling. All function names, structures, memory offsets, and implementation details have been sanitized and replaced with virtual/mock identifiers such as `sub_***` and `0xDEAD*` to comply with responsible disclosure practices and protect proprietary software integrity. No production credentials, user accounts, or third-party services are targeted or disclosed.

1. Abstract

This portfolio presents a controlled reverse-engineering and security analysis of a highly stripped native C++ mobile game binary that uses a custom data-obfuscation mechanism to protect internal application values.

The investigation focused on understanding how the application stores, retrieves, decrypts, and routes sensitive client-side values through its native execution layer. Because the binary contained no meaningful debugging symbols, the analysis relied primarily on string discovery, cross-reference analysis, control-flow tracing, register tracking, and instruction-level inspection.

The investigation identified a centralized obfuscation architecture built around a core 8-byte bitwise XOR routine, provisionally designated `sub_DECRYPT_GLOBAL`. This routine appeared to function as a common processing point for multiple application values.

Further tracing identified another important routine, designated `sub_ROUTER_MAIN`, which acted as a central data-abstraction and routing layer. The routine processed an item or variable

identifier through a large branch structure and determined how individual values were retrieved or processed.

The combination of these two components created a relatively centralized trust boundary. Once the relevant execution paths were reconstructed, it became possible to demonstrate, in a controlled research environment, how localized instruction modification could affect multiple logical values without requiring separate analysis of every individual data structure.

The analysis therefore highlights an important security principle, client-side cryptographic or obfuscation mechanisms should not be considered a sufficient security boundary when the client itself controls both the protected data and the code responsible for interpreting that data.

2. Static Analysis & Architectural Tracing

2.1 Target Architecture

The analyzed component was a native mobile game containing a heavily stripped C++ binary.

Platform	Mobile / Native Application
Architecture	ARM64 / AArch64
Binary Type	Native shared library
Implementation	Native C++
Symbol State	Heavily stripped
Function Naming	Generic sub_XXXXXX identifiers
Primary Analysis	Static reverse engineering
Data Protection	Custom XOR-based obfuscation

The absence of meaningful symbols significantly increased the analytical workload. As a result, the investigation relied on a combination of:

- String discovery
- Cross-reference analysis

- Control-flow graphs
- Function-call tracing
- Register tracking
- Data-flow analysis
- Instruction-level inspection
- Branch and switch-case analysis

2.2 String Anchoring & Core Decryption Discovery

The first stage of the investigation focused on identifying meaningful string literals and data references inside the stripped binary.

Because the original debugging symbols had been removed, internal routines could not be identified through descriptive function names. Instead, functions were represented by generic identifiers such as:

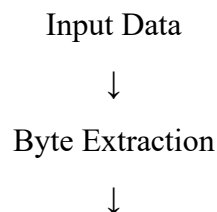
- sub_DECRYPT_GLOBAL
- sub_ROUTER_MAIN

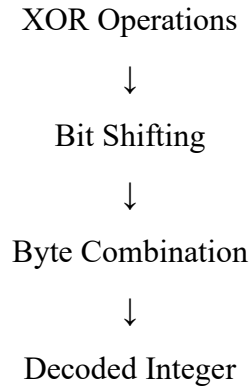
and similar address-based names.

String references were therefore used as initial analytical anchors. By following cross-references from relevant data structures and tracing their callers, several routines associated with runtime data processing were identified.

One of the most important discoveries was a centralized routine provisionally designated sub_DECRYPT_GLOBAL

The routine appeared to perform an 8-byte data transformation using a sequence of byte-level operations. A simplified representation of the observed processing model was:





At the instruction level, the routine contained operations conceptually equivalent to:

- LDRB W8, [X0,#2]
- LDRB W9, [X0,#5]
- EOR ...
- LSL ...
- ORR ...

The important observation was not the individual XOR operation itself, but the centralized nature of the implementation.

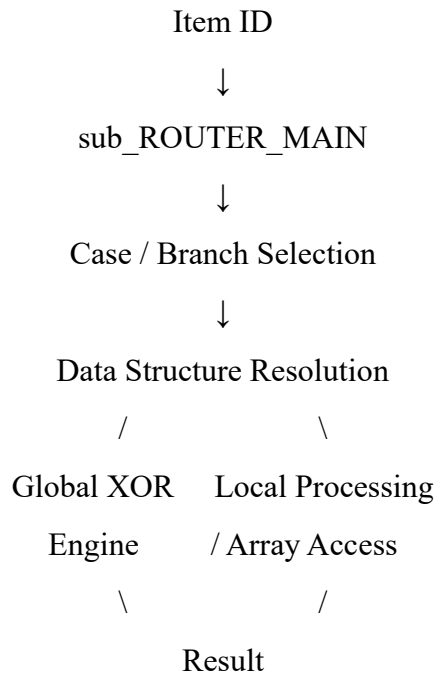
Instead of each data type having an independent protection mechanism, multiple logical values appeared to pass through the same processing routine. This made the routine an important architectural chokepoint for further analysis.

2.3 Identification of the Master Database Router

After identifying sub_DECRYPT_GLOBAL, the next objective was to determine how different application values reached the routine.

Cross-reference analysis revealed a larger abstraction function provisionally designated sub_ROUTER_MAIN

The function appeared to receive an item or variable identifier through register W0 and then use this identifier to select the appropriate processing path. The general execution model could be represented as:



The routine contained a large switch-like branch structure in which different identifiers were associated with different data locations or processing paths. This was particularly important because it demonstrated that the application's data abstraction layer was relatively centralized.

From an auditing perspective, this means that understanding the router could provide a much broader view of the application's internal data model than analyzing individual values independently.

3. Vulnerability Exploitation & Assembly Reengineering

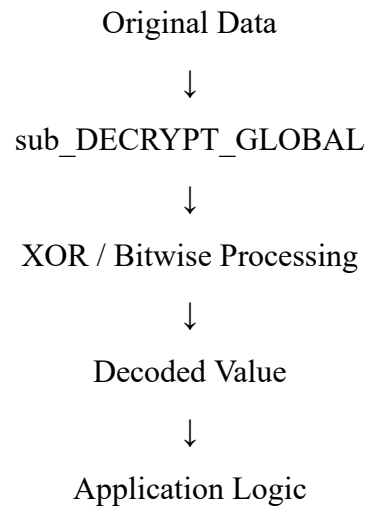
3.1 Centralized Data-Processing Weakness

The next phase of the investigation examined whether the centralized architecture could be influenced through localized instruction-level modification.

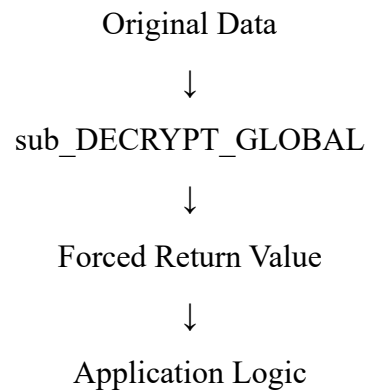
The primary research target was `sub_DECRYPT_GLOBAL`, because multiple logical values appeared to rely on this routine, modifying its return behavior could potentially affect multiple consumers simultaneously.

The objective was not to recover individual obfuscation keys for every data type, but to evaluate whether the centralized processing point itself represented a security weakness.

- This produced the following conceptual model:



- A modified execution path could instead be represented as:



3.2 Tactic A, Global Value Substitution

The most direct research modification targeted the entry portion of the centralized decryption routine.

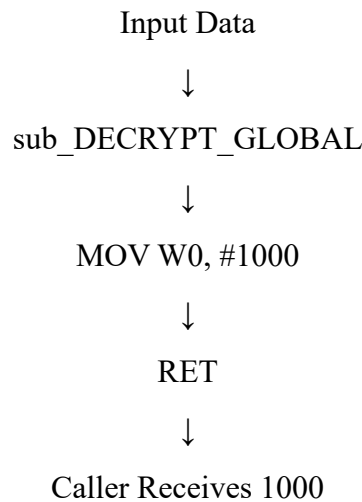
The original instruction sequence was represented in the sanitized research documentation as:

- .text:0x0012A100 LDRB W8, [X0,#2]
- .text:0x0012A104 LDRB W9, [X0,#5]

For controlled analysis, the execution path was conceptually replaced with:

- `.text:0x0012A100 MOV W0, #1000`
- `.text:0x0012A104 RET`

The purpose of this modification was to demonstrate what would happen if the centralized data-processing routine stopped performing its normal decoding operation and instead returned a predetermined value. The resulting execution model became:



The experiment demonstrated that a centralized client-side processing function could potentially become a single manipulation point for multiple consumers.

Rather than individually modifying every stored value, the research model showed that changing the common processing layer could influence the values returned to higher-level application logic.

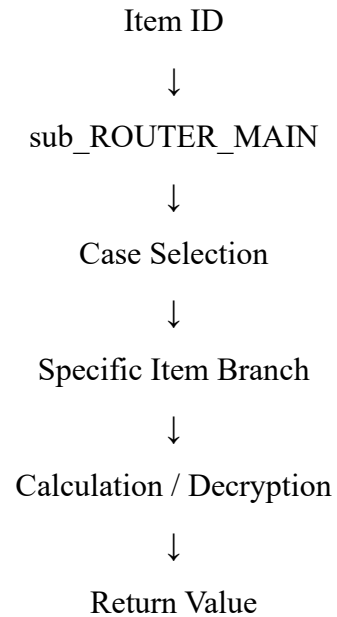
This represents a significant architectural weakness when the client assumes that the output of such a routine is trustworthy.

3.3 Tactic B, Runtime State Freezing

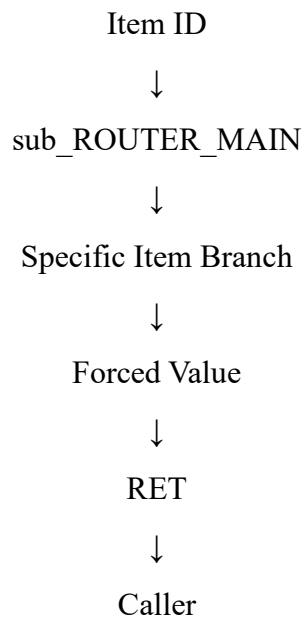
The second research direction focused on the routing layer represented by `sub_ROUTER_MAIN`, the purpose was to determine whether individual branches could be isolated from the global processing mechanism. Instead of modifying the entire decryption routine, individual logical cases within the router were examined.

The research process involved mapping individual branches and identifying where execution eventually returned a value to the caller.

- The generalized flow was:



- The modified model became:



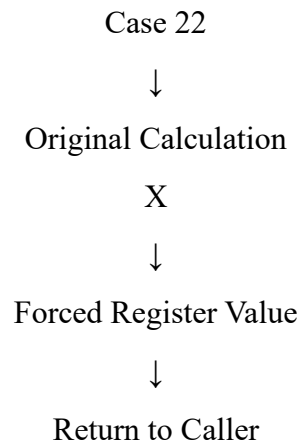
3.4 Premium Currency Chamber — Case 22

One of the isolated branches was represented by the sanitized research case Premium Currency Chamber — Case 22

The original branch contained calculations associated with the selected item type. For research purposes, the branch was conceptually modified using:

- `.text:0x0028F480 MOVZ W0, #12, LSL #16`
- `.text:0x0028F484 NOP`

The purpose of this modification was to replace the normal calculation path with a predetermined register value. The resulting execution model became:

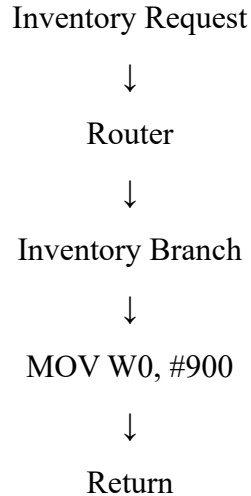


3.5 Inventory Cap / Milestone Override

A second isolated branch was associated with inventory limits or milestone-related values. The sanitized research modification was represented as:

- `.text:0x0028F4A4 MOV W0, #900`
- `.text:0x0028F4A8 NOP`

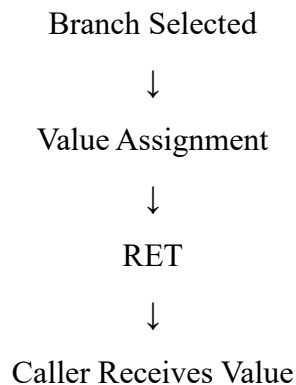
The purpose was to replace the original offset calculation and return a controlled value instead. The conceptual flow became:



3.6 Control-Flow Short-Circuit Interception

The final stage of the branch-level analysis focused on control-flow behavior. A sanitized early-return point was represented as `.text:0x0028F4CC RET`

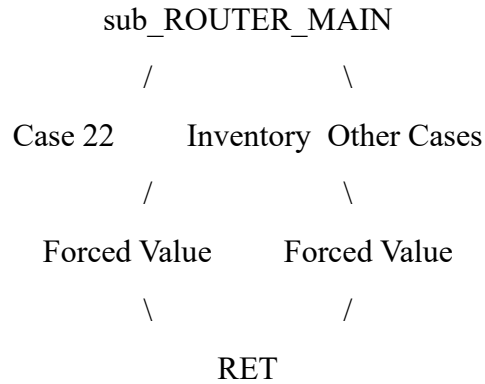
The purpose of the modification was to terminate the selected branch before execution reached the normal downstream processing logic. The resulting flow was:



This demonstrates an important reverse-engineering concept, the security impact does not always come from breaking the cryptographic operation itself. In some architectures, controlling the execution path around the cryptographic routine may be enough to change the application's behavior.

3.7 Runtime State Freezing Model

Combining the previous observations, the resulting research model can be represented as:



Under normal execution, the application would continue through its normal calculation, lookup, or decryption logic. Under the modified research model, selected branches instead returned predetermined values.

This created what can be described as a runtime state-freezing condition, where the application's higher-level logic could repeatedly receive the same client-controlled value.

4. Recommendations & Mitigations

The findings demonstrate that custom client-side obfuscation can increase the difficulty of casual inspection, but it cannot establish a reliable security boundary by itself. The following recommendations are intended for developers seeking to strengthen native mobile game against comparable reverse-engineering and binary-tampering techniques.

4.1 Avoid Centralized Decryption Chokepoints

A single global routine responsible for decoding many unrelated security-sensitive values creates an attractive target for reverse engineers. If the same routine processes currency, inventory, progression, and other sensitive values, compromising the routine may affect multiple logical components simultaneously.

Developers should therefore avoid designing critical application state around a single centralized decoding bottleneck. Where practical, sensitive operations should be separated according to their security requirements and validated independently.

4.2 Strengthen Control-Flow Protection

Large switch-case structures and predictable branch layouts can make static analysis significantly easier. Control-flow obfuscation can increase the cost of reconstructing the application's logical structure. Potential defensive techniques include:

- Control-flow obfuscation
- Selective control-flow flattening
- Symbol stripping
- Sensitive-string protection
- Runtime reconstruction of sensitive constants
- Removal of unnecessary diagnostic information

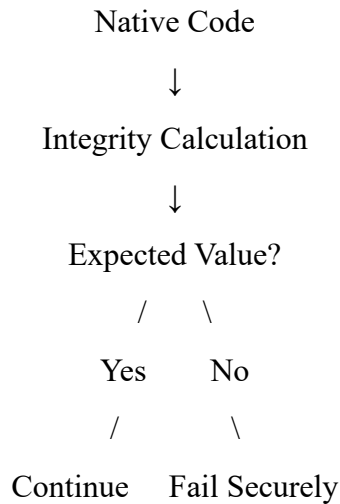
4.3 Protect Sensitive Runtime Values

Sensitive values should not be maintained as simple, predictable integers whenever the application architecture allows stronger protection. A defensive representation could use dynamically encoded values for example $\text{EncodedValue} = \text{Value} \text{ XOR } \text{RuntimeKey}$ with corresponding runtime decoding $\text{Value} = \text{EncodedValue} \text{ XOR } \text{RuntimeKey}$

The key should ideally be generated or derived dynamically rather than stored as a single static constant inside the binary. However, this mechanism should not be considered a replacement for server-side validation. An attacker controlling the client may ultimately observe or manipulate the decoding process.

4.4 Runtime Integrity Verification

Native code containing security-sensitive routines should be protected against unauthorized modification. A defensive integrity mechanism could periodically verify selected executable regions:



This can make localized instruction modification more difficult to perform without detection. Possible mechanisms include:

- Code-section integrity checks
- Runtime checksum verification
- Anti-tampering mechanisms
- Detection of unexpected executable modifications
- Verification of security-sensitive native modules

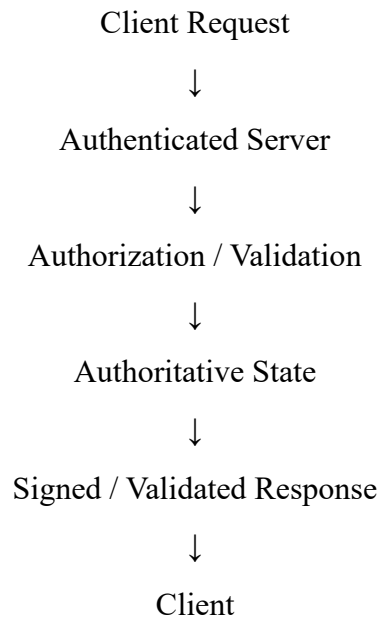
4.5 Move Critical State to the Server

The most important architectural mitigation is to avoid treating the mobile client as the final authority for security-sensitive state. Values such as:

- Premium currency
- Subscription entitlements
- Account-level inventory

- Purchases
- Rewards
- Transaction history
- Competitive progression

should preferably be validated by an authoritative backend. A stronger architecture can be represented as:



Under this architecture, modifying a local native routine would not be sufficient to permanently change the authoritative account state. The client would still be responsible for rendering and presenting information, but the backend would determine whether the requested transaction is actually valid.

5. Conclusion

This case study demonstrates how systematic static reverse engineering can expose weaknesses in a native application's client-side data-protection architecture. The analysis identified a centralized XOR-based processing routine, provisionally designated `sub_DECRYPT_GLOBAL`, together with a larger routing function designated `sub_ROUTER_MAIN`.

The investigation showed that the application's reliance on these centralized client-side routines created multiple opportunities for instruction-level manipulation. At the global level, modifying the common processing routine demonstrated how a predetermined value could be returned instead of the normal decoded result.

At the routing level, individual branches could be isolated and modified to return controlled values before reaching their original downstream logic. The research therefore demonstrates an important distinction between obfuscation and security.

A custom XOR routine, stripped symbols, control-flow complexity, and runtime integrity checks may all increase the cost of reverse engineering. However, if the client remains responsible for determining the authoritative value of a security-sensitive asset, an attacker who gains sufficient control over the client may eventually manipulate that state.

The fundamental security principle remains, the mobile client must be treated as an untrusted environment. Obfuscation can slow down reverse engineering. Anti-tampering can make modification more difficult. Runtime encoding can make memory inspection less convenient. However, none of these mechanisms can guarantee the integrity of a value that is ultimately controlled by the user's device.

Overall Assessment, the analyzed architecture demonstrates a meaningful client-side trust-boundary weakness. The centralized data-processing design reduces the amount of code an attacker needs to understand before influencing multiple logical values, while the router provides additional localized manipulation points. Strengthening the native client remains valuable as a defense-in-depth strategy.